

MODULE 4 · WEEK 2

TSFresh — Automated Time-Series Features

Let the data speak: auto-extract hundreds of statistical features, filter to the ones that predict, and classify robot failures.

Lectures 43–44 · 46–48



This week

- 1 **EDA for time series** — what to look at first
- 2 **TSFresh** — automatic feature extraction
- 3 **Feature selection** — keep only what predicts
- 4 **Don't leak!** — extract within the right boundaries
- 5 **GitHub Actions** — automate the data sync



From manual windows to automatic features

- Last week you **chose** the lags by hand — powerful, but manual.
- **TSFresh** takes the opposite approach: extract hundreds of statistics automatically.
- Mean, variance, entropy, autocorrelation, peaks... then let selection find the useful ones.

Key idea The window method is hand-picked; TSPresh is exhaustive then filtered.



Intro: robot failures & TS EDA

- Case study: classify **robot failures** from sensor time series.
- Start with EDA — plot the sensor traces, compare failing vs. healthy runs.
- Each labeled time segment becomes one example to classify.

TSFresh in action

- Point TSFresh at your long-format series (id, time, value) and it computes a rich feature table.
- Hundreds of features per series — far more than you'd engineer by hand.
- Then feed the table into any standard sklearn classifier or regressor.

```
from tsfresh import extract_features
X = extract_features(df, column_id='id',
                    column_sort='time')
```

Feature selection — and don't leak!

- TSFresh's selection keeps only features statistically related to the target — cutting noise.
- **Critical:** fit selection on training data only, then apply to test.
- Extracting or selecting across the whole dataset leaks the future into your features.

```
from tsfresh import select_features  
X_train_sel = select_features(X_train, y_train)
```

Key idea TSFresh is powerful but easy to leak with — keep extraction and selection inside the training boundary.



TSFresh trade-offs

- Powerful and thorough, but **slow** — hundreds of features over many series adds up.
- Great when you want to be exhaustive and let the data decide.
- Use the window method when you need speed or interpretable, hand-chosen lags.

Automate the data sync with GitHub Actions

- **GitHub Actions** runs code on a schedule or on every push — no laptop required.
- Use it to pull fresh data, re-run extraction, and commit the updated dataset automatically.
- Your pipeline stays current without anyone remembering to run it.

```
# .github/workflows/sync.yml
on:
  schedule:
    - cron: '0 6 * * *'    # daily at 06:00 UTC
```



Lab this week

- Use TSFresh to generate and select features from raw time series, then classify with a standard sklearn model.
- Mind the speed/power trade-off, and keep extraction leak-free.



Key takeaways

- TSFresh auto-extracts hundreds of time-series features, then filters to what predicts.
- It complements the manual window method — exhaustive vs. hand-picked.
- Extract and select inside the training boundary to avoid leakage.
- It's thorough but slow — choose the tool that fits the job.
- GitHub Actions automates the data sync so the pipeline runs itself.